

One Grid, Every Function

An 8×8 image pushed through the whole simulator, step by step

This document shows one 8×8 image travelling through every function in `mri/`. Every figure was produced by actually running that code; the script is `docs/walkthrough/make_figs.py`, so if you change a function you can regenerate the pictures and see what your change did.

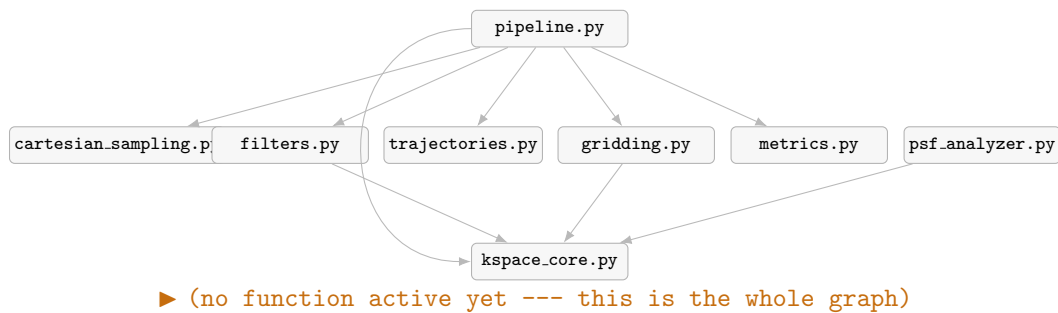
Two colour conventions run through the whole document:

Grey panels: image space — pixel values.

Magma panels (black $\rightarrow$ purple $\rightarrow$ yellow): k -space — frequency values.

Whenever the colour changes, you have crossed between the two domains. The number printed in each cell is the actual array value. The flow diagram before each step shows the eight files and highlights the one whose function is running at that moment; arrows point from a file to the file it imports from.

1 The map: who calls whom



This diagram shows the eight files and their import relationships. Read each arrow as “needs”.

1. `pipeline.py` (top).

Its Job: it is the coordinator — the one function the GUI calls.

The Action: it imports from every other module and runs them in the correct order.

2. The six middle modules.

Depend on `kspace_core`:

`filters` — for `radius_grid`

`gridding` — for `from_kspace`

`psf_analyzer` — for both

Stand alone: `cartesian_sampling`, `trajectories`, `metrics` — pure NumPy.

3. `kspace_core.py` (bottom).

Its Job: the only place the FFT is ever called. Every trip into or out of k -space passes through it.

File	Domain	Job in one line
kspace_core	both	the only place the FFT is called
cartesian_sampling	k -space	builds 0/1 masks
filters	k -space	fades the edges; adds noise
trajectories	k -space	lists non-Cartesian coordinates
gridding	k -space	scattered points $\leftrightarrow$ grid
psf_analyzer	image	what artifact a mask will cause
metrics	image	scores a reconstruction
pipeline	both	runs the chain in the right order

What one call actually does

When the GUI calls `pipeline.reconstruct(img, {"sampling":"radial"})`, this is the sequence of functions that execute, in order:

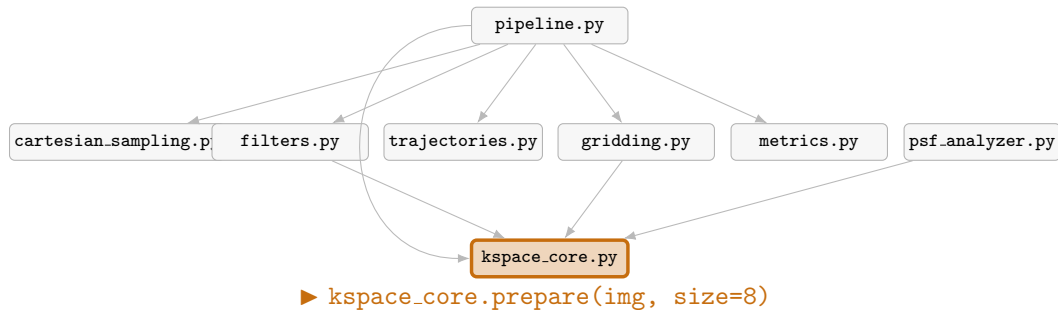
```

pipeline.reconstruct
  kspace_core.to_kspace image -> centred k-space
  trajectories.radial spoke coordinates + density weights
  gridding.sample_at read k-space at those coordinates
  gridding.grid_accumulate spread the samples back onto the grid
  filters.apply_window (window = "none" here, so multiply by ones)
  filters.add_noise (noise = 0.0 here, so return unchanged)
  kspace_core.from_kspace gridded k-space -> image
  kspace_core.log_magnitude k-space for display
  metrics.error_map |original - recon|
  metrics.compute_metrics mse, psnr, ssim

```

The rest of this document walks that list one function at a time.

2 The input



This step shows the image we will use for everything that follows. It is built from two parts:

1. The block (rows 2–4, columns 1–3).

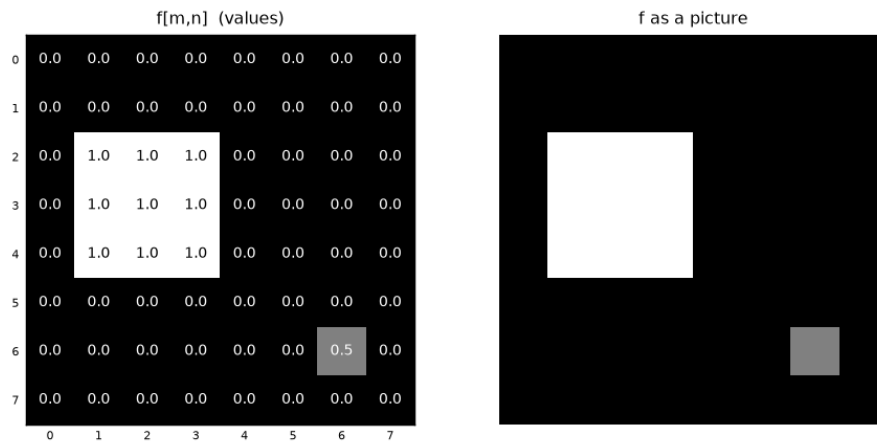
What it is: nine pixels of value 1.0, sitting off-centre.

Why it is there: it has an edge in both directions, so every direction of k -space gets something to measure.

2. The impulse (row 6, column 6).

What it is: one lone pixel of value 0.5.

Why it is there: a single bright point has energy at *every* frequency, which makes ghosts and blur easy to spot later.



```
f = np.zeros((8, 8))
f[2:5, 1:4] = 1.0 # the block
f[6, 6] = 0.5 # the impulse
f = prepare(f, size=8)
```

Reading the figure

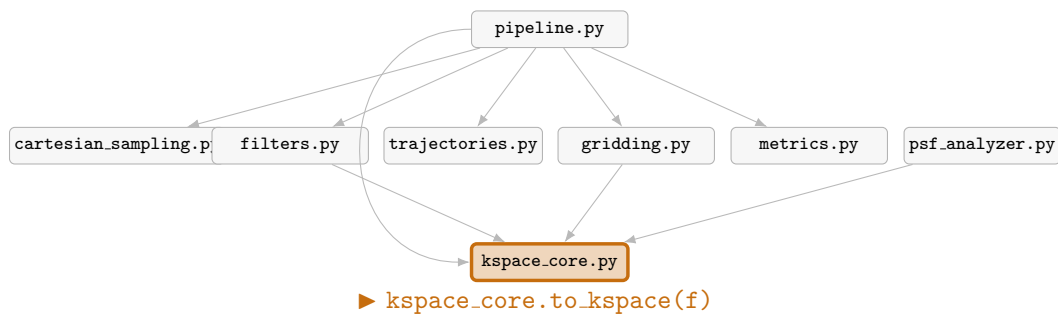
Left panel: the 64 values. Everything not in the block or the impulse is exactly 0.

Right panel: the same array drawn as a picture. This is what a scanner would be trying to recover.

The number to remember: the pixel sum is $9 \times 1.0 + 0.5 = \mathbf{9.5}$. It reappears in the very next step.

(Note: *prepare* is the same input path the real app uses — grayscale, pad, resize, normalise. On an input that is already 8×8 and already in $[0, 1]$, every stage except normalise is a no-op, so f passes through unchanged.)

3 Forward: image to k -space



This step shows the forward transform. It is two NumPy calls, and each one does a different job:

1. fft2 — the transform itself.

Its Job: for each of the 64 possible waves (u, v) , ask “how much of this wave is in f ?”

The Action: it computes the sum in the box below for every (u, v) and returns a 8×8 complex matrix. The answer for $u = v = 0$ lands in the *top-left corner*.

2. fftshift — the re-tiling.

Its Job: move the zero-frequency entry from the corner to the centre so “distance from centre = frequency” holds.

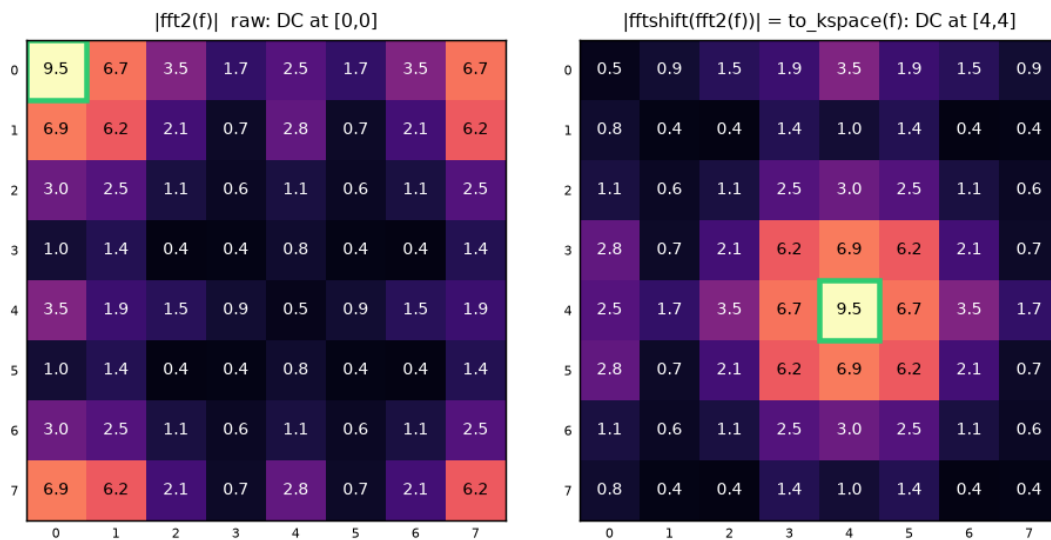
The Action: it swaps the four quadrants diagonally. No value changes; only positions do.

The maths

$$K[u, v] = \sum_{m=0}^7 \sum_{n=0}^7 f[m, n] e^{-i2\pi(\frac{um}{8} + \frac{vn}{8})}$$

At $u = v = 0$ the exponential is 1 and the double sum collapses to “add up every pixel” — the DC term.

```
def to_kspace(img):
    return np.fft.fftshift(np.fft.fft2(img))
```



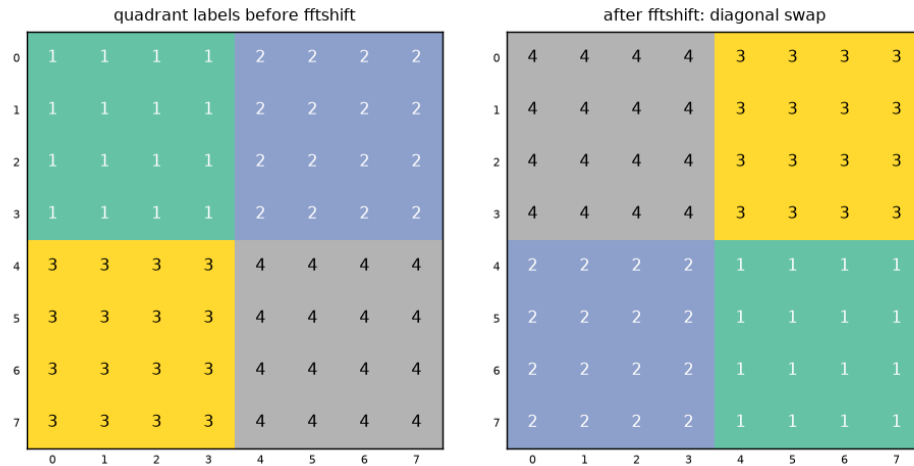
Reading the figure

Left panel (after `fft2` only): the green cell at $[0, 0]$ reads 9.5 — the pixel sum, exactly as the maths predicts. The large values hug the corners and edges, because the negative frequencies are wrapped round to the far side.

Right panel (after `fftshift`): the same 9.5 is now at $[4, 4]$, and values fall off in every direction as you move away from it.

What changed: nothing except position. Both panels contain the same 64 numbers.

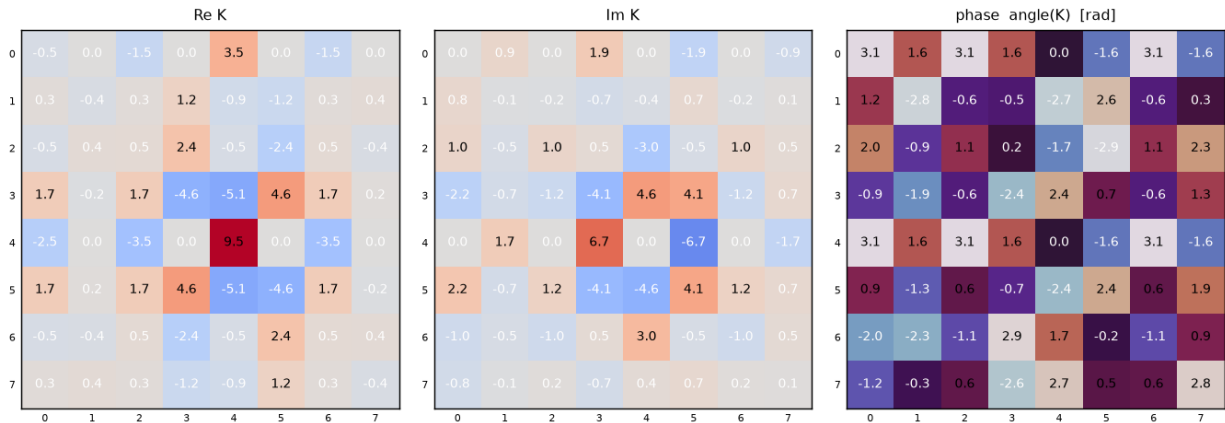
What fftshift actually moves



1. **Quadrant 1** $\leftrightarrow$ **Quadrant 4**. Top-left (holding the DC corner) trades places with bottom-right.
2. **Quadrant 2** $\leftrightarrow$ **Quadrant 3**. Top-right trades with bottom-left.

After the swap the four corners that held the low frequencies meet in the middle. `ifftshift` is the same operation run backwards.

k -space is complex



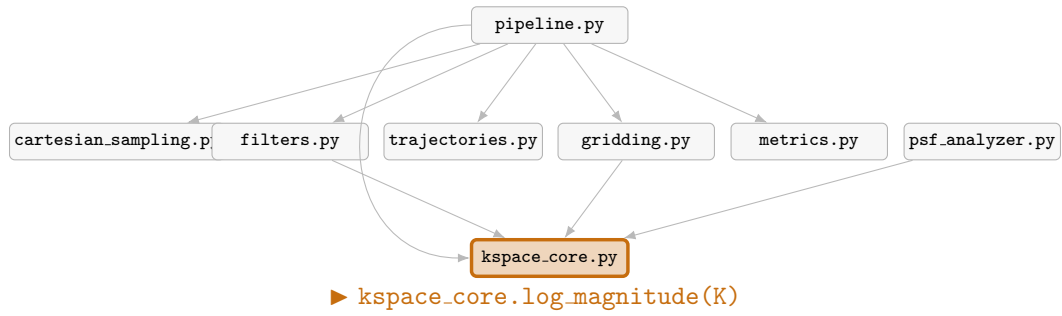
Each entry $K[u, v]$ is $a + bi$, and the two halves carry different facts:

Magnitude $|K|$ (what the earlier panels showed): *how much* of that wave is present.

Phase $\angle K$ (right panel): *where* that wave sits. Shift the block one pixel right and the magnitude panel would not change at all — only the phase would.

(Note: the real and imaginary panels are shown for completeness. In practice you read magnitude and phase; nothing downstream looks at Re and Im separately.)

4 Making k -space visible



This step shows the display helper. It has one job and one rule:

1. The compression.

Its Job: squeeze a huge dynamic range into something a screen can show.

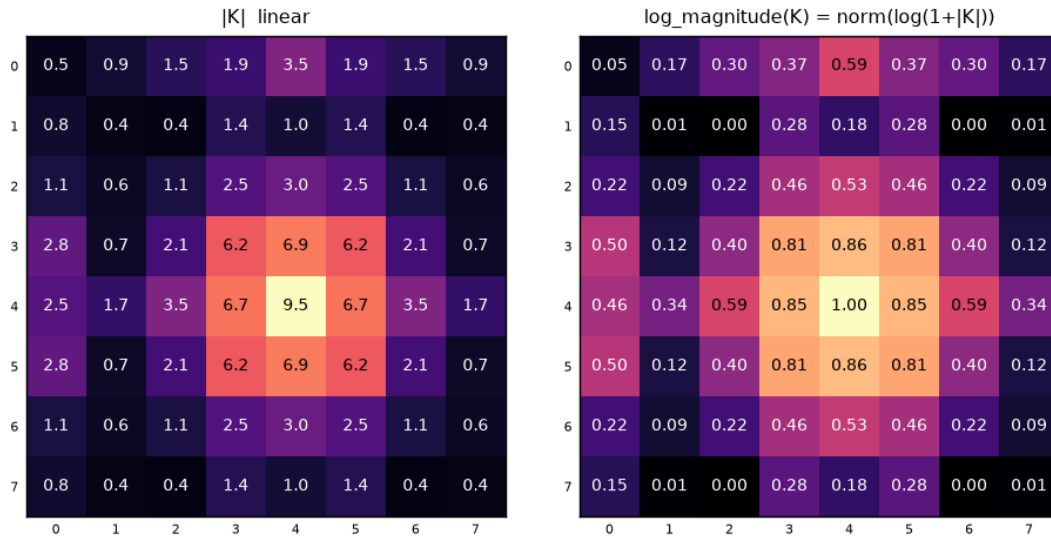
The Action: take $\log(1 + |K|)$ and rescale to $[0, 1]$. The $1 +$ keeps $\log 0$ from ever happening.

2. The rule.

Display only. The output is no longer the recipe for the image. It is never passed to `from_kspace`.

The maths

$$V[u, v] = \frac{\log(1 + |K[u, v]|)}{\max \log(1 + |K|)}$$



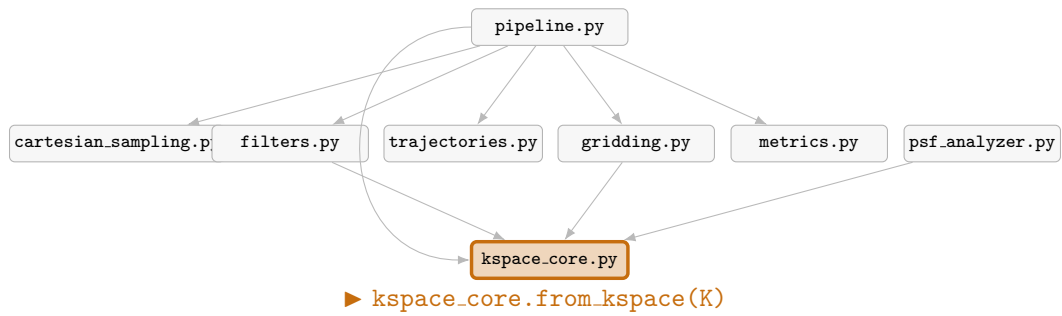
Reading the figure

Left panel: linear $|K|$, range 0.4 to 9.5.

Right panel: after the log. The dark cells have been lifted, the bright centre is still brightest, but the gap between them is much smaller.

Why it looks mild here: on 8×8 the range is only about $20\times$. On a 256×256 phantom the centre is a *million* times the edges, and without the log you would see one white dot on black.

5 Inverse: k -space back to image



This step shows the inverse transform. It is the forward transform run backwards, in three calls that mirror the two from Section 3:

1. `ifftshift` — undo the re-tiling.

The Action: swap the quadrants back so the DC term returns to the corner, which is where `ifft2` expects it.

2. `ifft2` — undo the transform.

Its Job: take the 64 “how much of each wave” answers and add all 64 waves back together.

The Action: it computes the sum in the box below. Sign flipped, divided by the number of pixels.

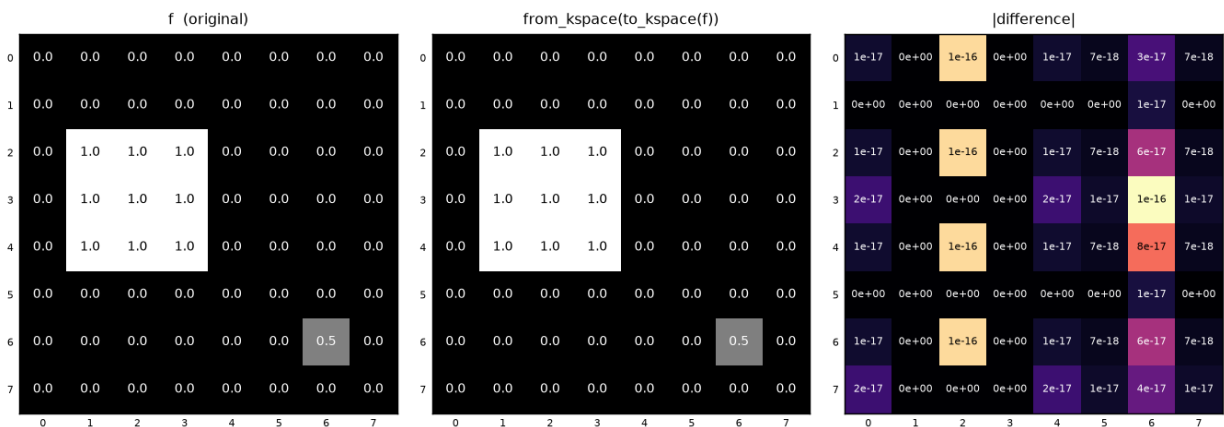
3. `abs` — drop the imaginary dust.

The Action: the result is complex with a tiny imaginary part from rounding; `abs` discards it and returns a real image.

The maths

$$f[m, n] = \frac{1}{64} \sum_{u=0}^7 \sum_{v=0}^7 K[u, v] e^{+i2\pi\left(\frac{um}{8} + \frac{vn}{8}\right)}$$

```
def from_kspace(k):
    return np.abs(np.fft.ifft2(np.fft.ifftshift(k)))
```



Reading the figure

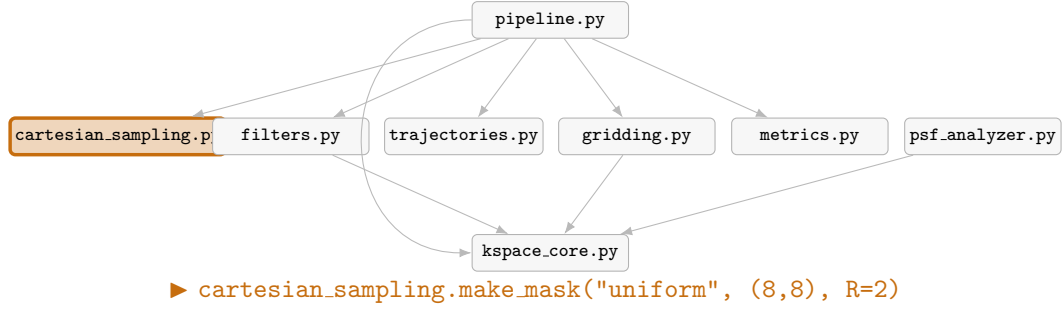
Left and middle panels: identical to the eye and to the printed values.

Right panel: the difference, cell by cell. Every entry is about 10^{-16} — floating-point

dust. The transform is lossless.

What this proves: Gate 1. With nothing changed in the middle, the sandwich returns exactly what went in.

6 Undersampling with a mask



This step shows the first thing that actually damages the image. Undersampling is one multiplication, and the mask is what decides which entries survive:

1. The mask S (left panel).

What it is: an 8×8 array of ones and zeros.

Row value 1 (kept): the scanner measured this row of k -space.

Row value 0 (skipped): the scanner never measured it, so the entry becomes 0.

The Action for uniform $R = 2$: keep every second row, starting from the row that contains DC.

2. The multiplication (middle panel).

The Action: $K_{\text{sub}} = K \cdot S$ element-wise. Kept rows pass through; skipped rows are zeroed.

3. The reconstruction (right panel).

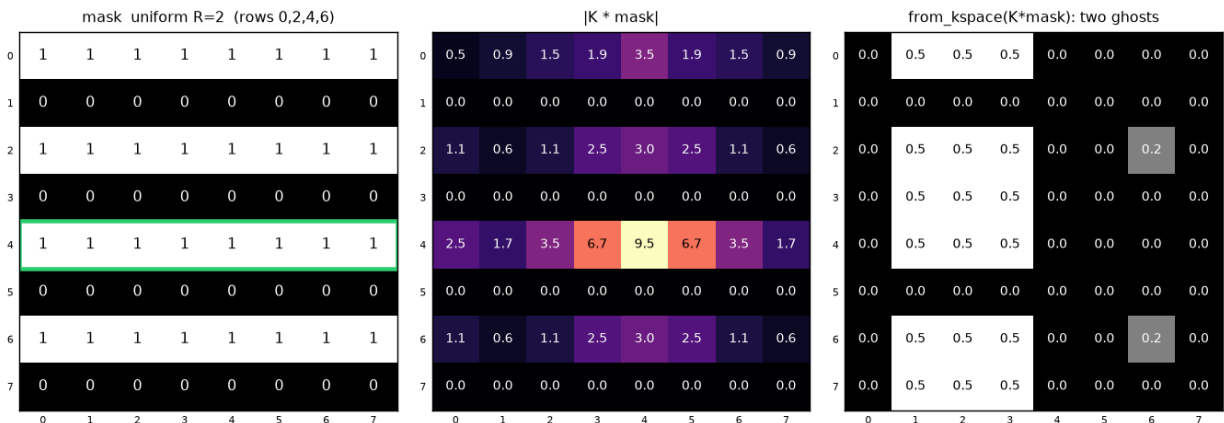
The Action: `from_kspace` on K_{sub} . With half the rows missing, the image comes back with a ghost.

The maths

$$K_{\text{sub}}[u, v] = K[u, v] \cdot S[u, v], \quad S[u, v] \in \{0, 1\}$$

```

rows, _ = shape
mask = np.zeros(shape, dtype=np.uint8)
mask[(rows//2) % R :: R, :] = 1 # anchored on the DC row
  
```



Reading the figure

Left panel: rows 0, 2, 4, 6 are kept — 32 of 64 samples, so $R_{\text{actual}} = 2.0$. The green outline is row 4, the DC row, kept because the slice starts at $(8//2) \% 2 = 0$.

Middle panel: the odd rows of K are simply gone.

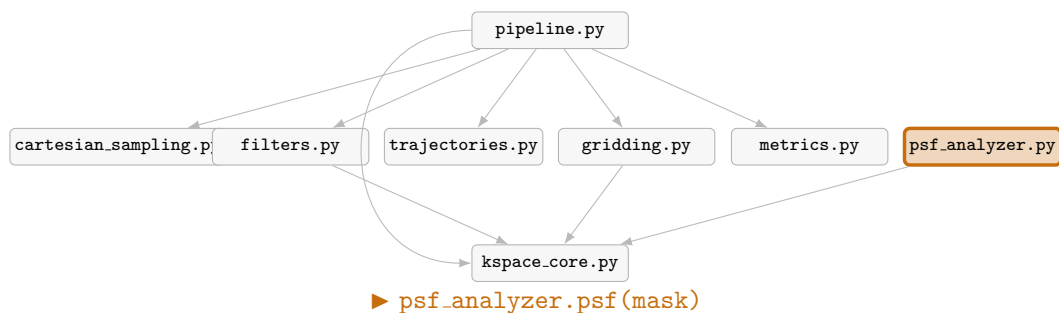
Right panel: the block appears **twice**, each copy at half brightness (0.5). The second copy is shifted down by $8/2 = 4$ rows and wraps round to rows 6, 7, 0. The impulse also has a ghost, at $[2, 6]$.

The rule this demonstrates: skip every R -th row and you get R copies, each $1/R$ as bright, spaced N/R rows apart. That is aliasing.

The bug this one line prevents

With `mask[:, :R]` instead of `mask[(rows//2) % R :: R]`, $R = 3$ would keep rows 0, 3, 6 and lose row 4 — the DC term — and the reconstruction would come out 40% too dark. The offset guarantees the centre row is always in the kept set, whatever R is. Day 1 found this the hard way.

7 Predicting the ghosts before reconstructing



This step shows the point spread function. It answers “what will this mask do to the image?” without needing the image at all:

1. The PSF itself (left panel).

Its Job: show what one bright pixel turns into after sampling with this mask.

The Action: inverse-transform the mask *alone* — no image involved — and centre the result.

2. The profile (right panel).

Its Job: put numbers on the spikes.

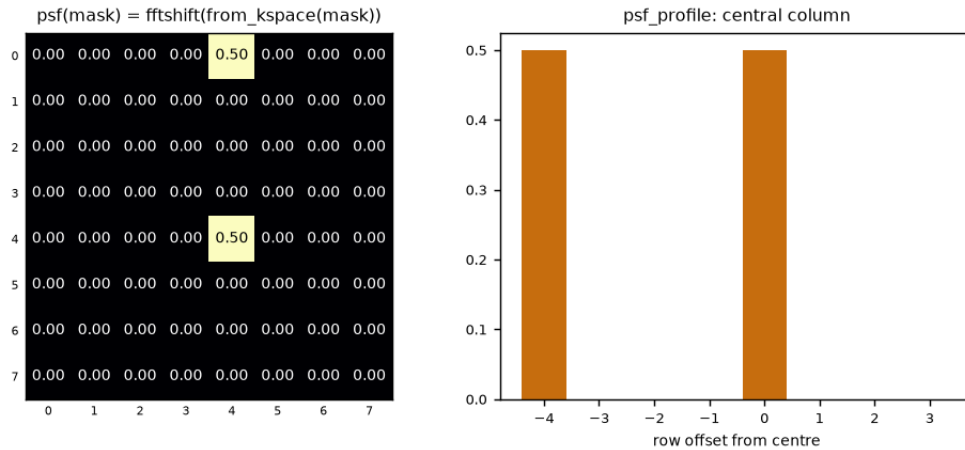
The Action: take the central column of the PSF and plot it as bars against row offset.

The maths

$$\text{PSF} = \mathcal{F}^{-1}\{S\}, \quad f_{\text{recon}} = f * \text{PSF}$$

The full reconstruction is the original convolved with the PSF. So whatever shape the PSF has, every pixel of the image gets stamped with that shape.

```
def psf(mask):
    return np.fft.fftshift(from_kspace(mask.astype(float)))
```



Reading the figure

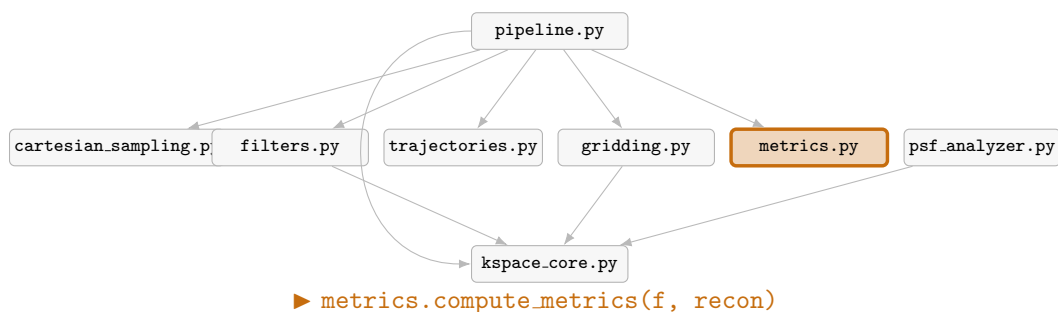
Left panel: two spikes of equal height, 0.5 each — one at the centre, one four rows away. Everything else is 0.

Right panel: the bars confirm it. `ghost_offsets` reports `[-4, 0]`.

The connection to Section 6: convolving the block with two half-height spikes four rows apart gives exactly two half-bright copies four rows apart. The PSF *is* the ghost, seen before the image was involved.

`peak_sidelobe = 1.0`: the ghost is as bright as the original. Uniform skipping always gives 1.0; random skipping would scatter it into grain.

8 Scoring the damage



This step shows how “that looks worse” becomes a number. Three scores are computed, and one map is drawn:

1. MSE — mean squared error.

The Action: average of $(f - \hat{f})^2$ over all 64 pixels. 0 is perfect.

2. PSNR — peak signal-to-noise ratio, in dB.

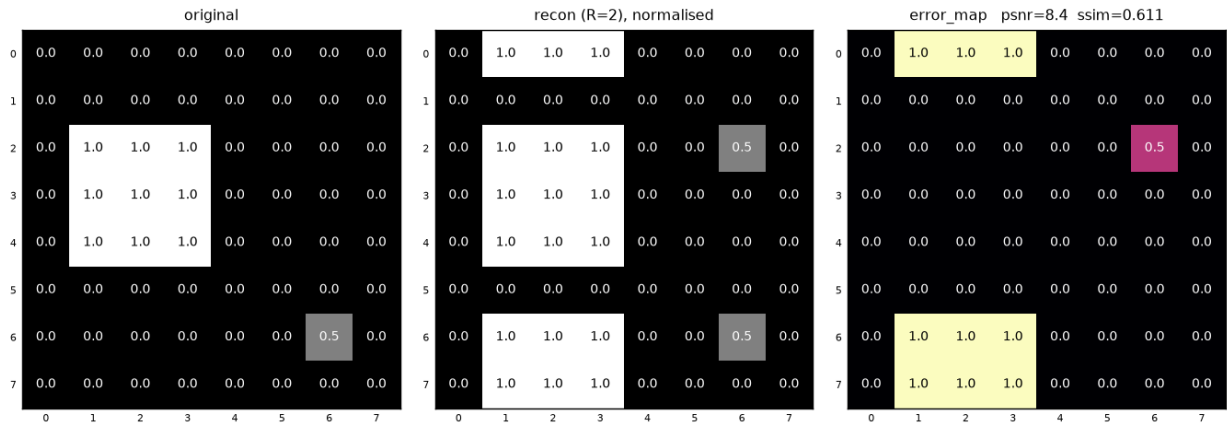
The Action: $10 \log_{10}(1/\text{MSE})$. Higher is better; infinite for an exact match, which the GUI has to be ready to print.

3. SSIM — structural similarity.

The Action: compares local structure rather than pixel-by-pixel differences. In $[0, 1]$, and tracks what the eye sees better than MSE does.

4. The error map (right panel).

The Action: $|f - \hat{f}|$ per pixel, for the GUI's heatmap.

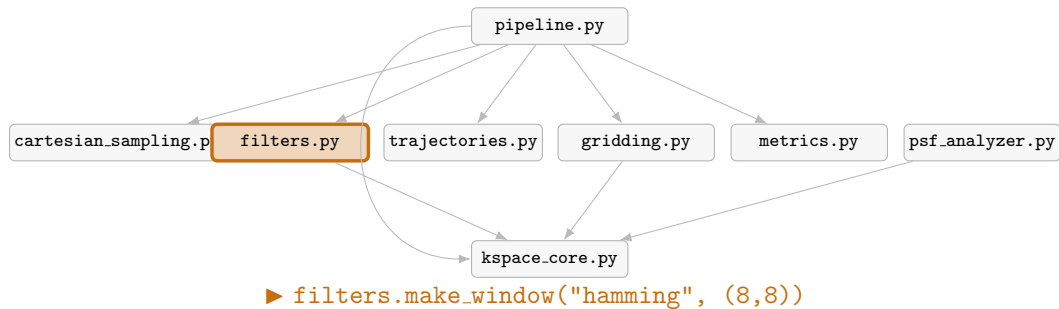


Reading the figure

Right panel: the error is brightest exactly where the ghost landed (rows 6, 7, 0) and where the original block lost half its brightness.

The readout: PSNR 8.4dB, SSIM 0.61. These are the numbers the GUI's status line will show when the slider is at $R = 2$.

9 Fading the edges: apodization



This step shows the apodization window. It replaces “chop the high frequencies off” with “fade them out”, and is built from three pieces:

1. The centred index $u = \text{np.arange}(8) - 4$.

Its Job: make $u = 0$ land at index 4, where DC sits after `fftshift`.

The Action: turns $[0, 1, \dots, 7]$ into $[-4, -3, \dots, 3]$.

2. The 1D raised cosine w_u .

The Action: $0.54 + 0.46 \cos(\pi u/4) \rightarrow 1.0$ at $u = 0$, falling to 0.08 at the edge.

3. The outer product $\text{np.outer}(w_u, w_v)$.

Its Job: turn two 1D curves into one 2D surface without a loop.

The Action: $W[i, j] = w_u[i] \cdot w_v[j]$. Peak 1.0 at the centre; corners $0.08 \times 0.08 = 0.006$.

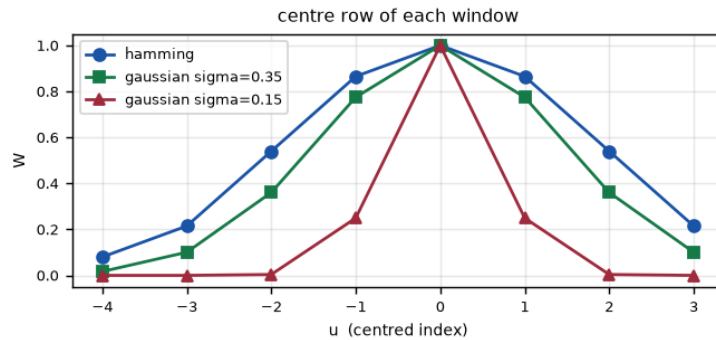
The maths

$$W[u, v] = \left(0.54 + 0.46 \cos \frac{\pi u}{4}\right) \left(0.54 + 0.46 \cos \frac{\pi v}{4}\right), \quad u, v \in \{-4, \dots, 3\}$$

```

u = np.arange(rows) - rows//2 # [-4,-3,-2,-1,0,1,2,3]
wu = 0.54 + 0.46*np.cos(np.pi*u/(rows/2))
W = np.outer(wu, wv) # W[i,j] = wu[i]*wv[j]

```

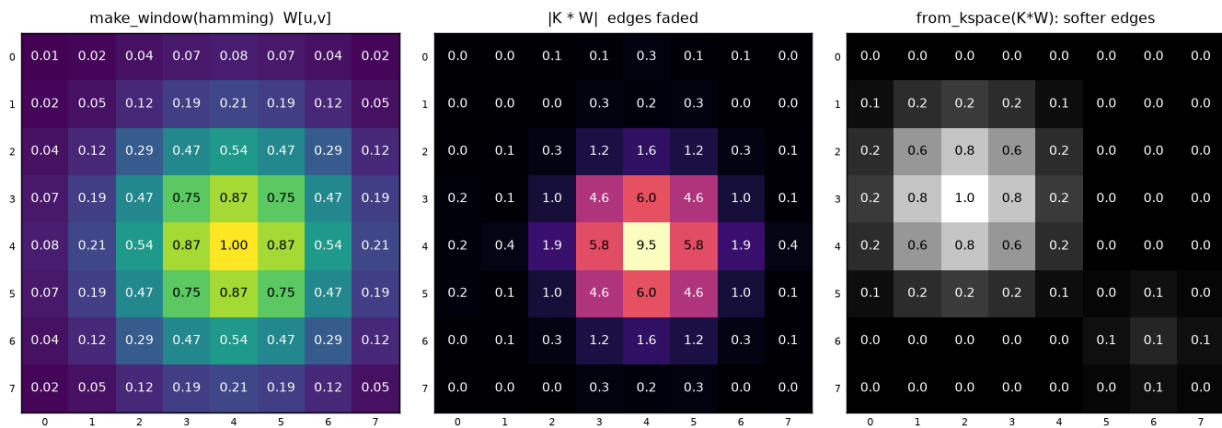


The three curves are the centre row of each window kind:

Hamming (blue): fixed shape, floors at 0.08 — deliberate, balancing ripple suppression against blur.

Gaussian $\sigma = 0.35$ (green): gentler at the centre, reaches near zero at the edge.

Gaussian $\sigma = 0.15$ (red): fades hard — most of k -space is nearly gone. Small σ means more blur. That σ becomes a slider.



Reading the figure

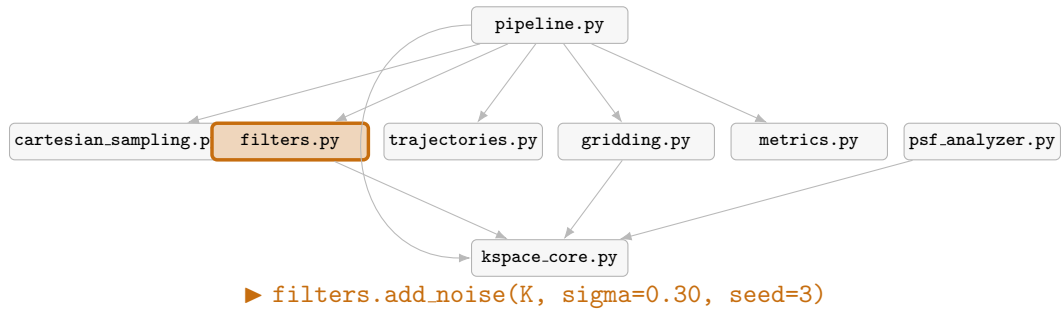
Left panel: the window. 1.00 dead centre, 0.08 on the edge row.

Middle panel: $|K \cdot W|$. The centre column is untouched; the outer values have been scaled toward zero rather than deleted.

Right panel: the reconstruction. The block's edges have softened — look at the 0.7–0.9 values bleeding out of it. Blur is the price of no ringing.

(Note: on an 8×8 grid there is no room for ringing to show, so all you see here is the blur side of the trade. On the phantom this same window is the difference between rippled edges and clean ones.)

10 Adding scanner noise



This step shows thermal noise being added. Two decisions inside the function matter:

1. Two independent noise draws.

Its Job: model the two quadrature channels a real receiver measures.

The Action: one Gaussian array is added to the real part, a separate one to the imaginary part.

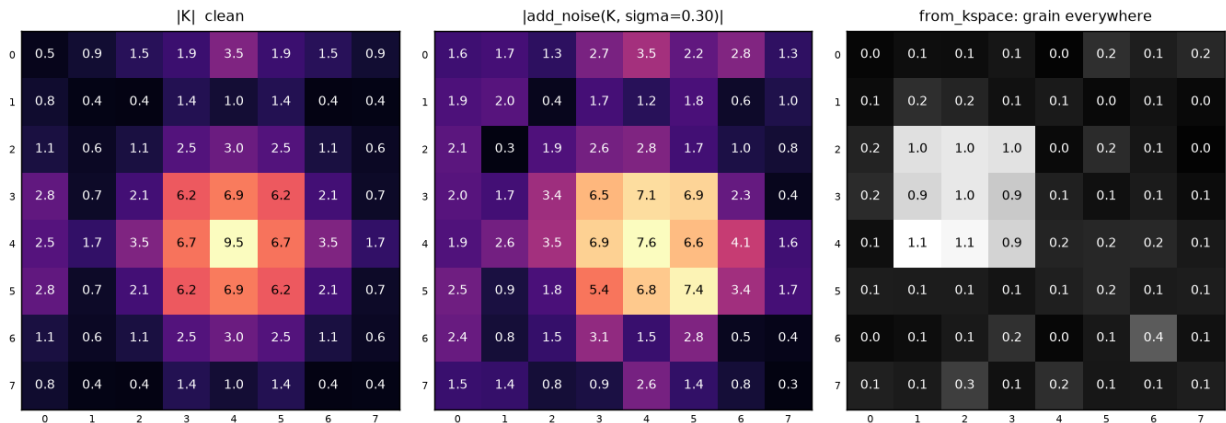
2. The scale $s = \sigma \cdot \overline{|K|}$.

Its Job: make the same slider value mean the same visible grain on any image.

The Action: σ is a fraction of the mean magnitude, not an absolute number. Here $\overline{|K|} = 2.2$, so $s = 0.66$.

The maths

$$K_{\text{noisy}} = K + \mathcal{N}(0, s^2) + i\mathcal{N}(0, s^2), \quad s = \sigma \cdot \overline{|K|}$$



Reading the figure

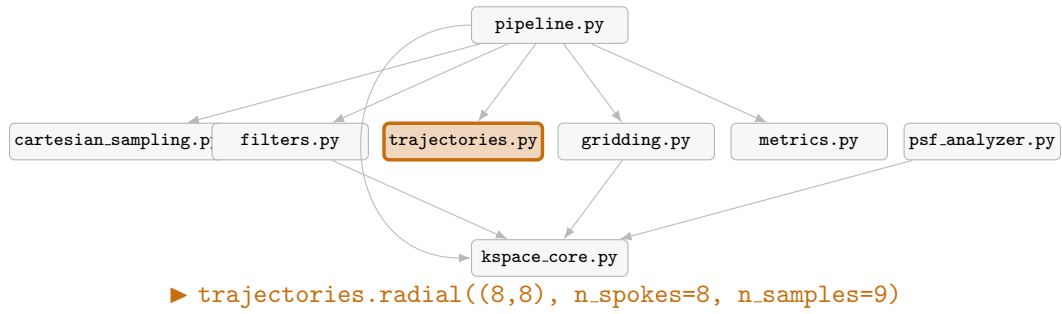
Left vs middle panel: compare any cell. Each has been nudged by roughly 0.66 in a random direction.

Right panel: grain everywhere, including the background that should be 0.

Why everywhere: every k -space entry contributes to every pixel, so noise in one place in k -space is noise spread across the *whole* image.

(Note: `sigma=0` returns the input object untouched rather than adding an array of zeros, so the GUI can leave noise switched off at no cost.)

11 Leaving the grid: radial spokes



This step shows the first non-Cartesian trajectory. Instead of a mask that picks grid rows, it produces a list of coordinates that are not on the grid at all. Four parts:

1. The radius vector $\mathbf{r}$.

The Action: 9 evenly spaced values from -4 to $+4$. One spoke runs from $-k_{\max}$ through the centre to $+k_{\max}$.

2. The angle vector θ .

The Action: 8 angles from 0 to π , not 2π — each spoke already covers both halves, so a full circle would retrace every spoke.

3. The outer products $\text{np.outer}(\sin(\theta), \mathbf{r})$ and $\text{np.outer}(\cos(\theta), \mathbf{r})$.

Its Job: build every (angle, radius) combination without a nested loop.

The Action: $8 \times 9 = 72$ points, each $(k_y, k_x) = (r \sin \theta, r \cos \theta)$, flattened into a $(72, 2)$ array.

4. The density weights `_ramp_weights`.

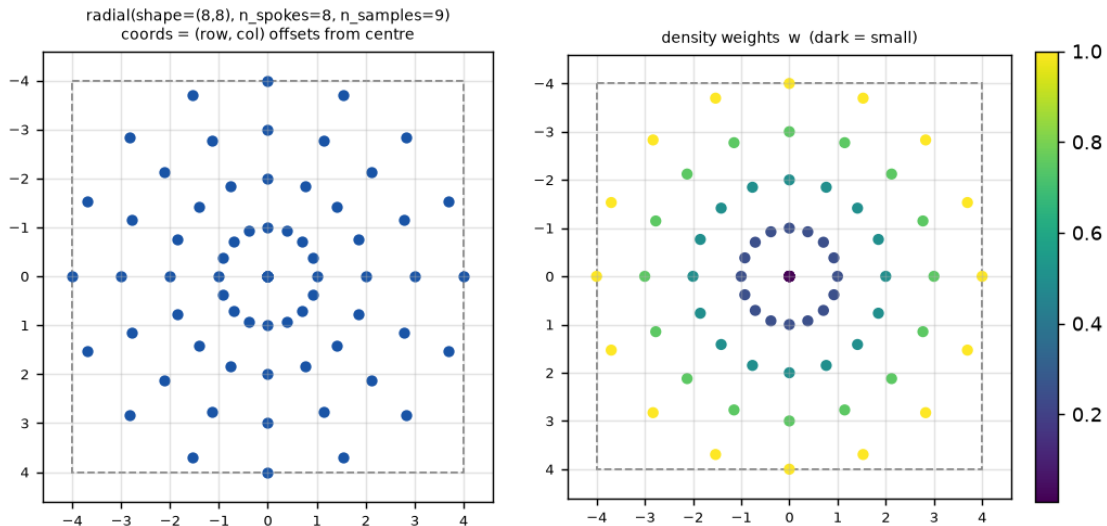
Its Job: pre-compensate for the crowding at the hub, where all eight spokes meet.

The Action: $w \propto |r|$, floored at a small positive value so the $r = 0$ sample is not zeroed out.

The maths

$$k_y = r \sin \theta, \quad k_x = r \cos \theta, \quad r \in [-4, 4], \quad \theta_j = \frac{j\pi}{8}, \quad j = 0..7$$

```
r = np.linspace(-4, 4, 9) # [-4, -3, ..., 3, 4]
theta = np.arange(8)*np.pi/8
ky = np.outer(np.sin(theta), r).ravel() # 8 x 9 = 72 points
kx = np.outer(np.cos(theta), r).ravel()
w = _ramp_weights(np.tile(r, 8), 8)
```



Reading the figure

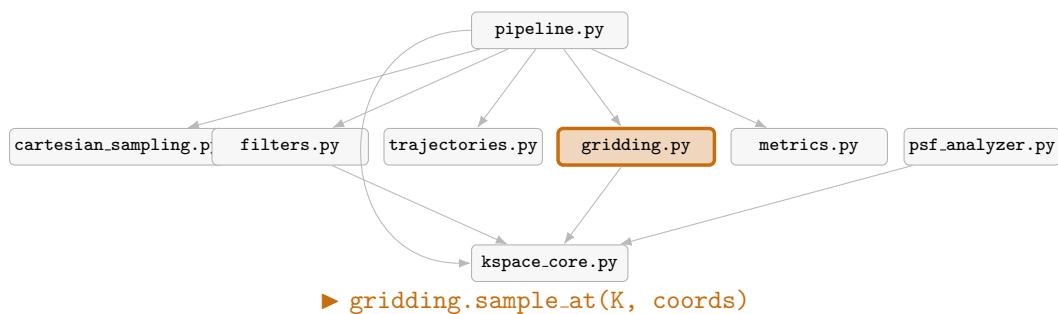
Left panel: the 72 coordinates form a wheel. The first spoke is horizontal: $(0, -4), (0, -3), \dots, (0, 4)$.

The crowding: every spoke passes through $(0, 0)$, so eight samples land on the same spot. That is what density compensation exists to fix.

Right panel: the weights along one spoke read $[1, .75, .5, .25, .01, .25, .5, .75, 1]$ — tiny at the hub, full at the rim.

(Note: these coordinates are offsets from the centre, not array indices. The centre is $(0, 0)$, not $(4, 4)$. Converting to indices is the gridding step's job, and keeping that conversion in one place avoids a whole family of off-by-four bugs.)

12 Reading k -space at the spoke points



This step shows the simulator pretending to be a radial scanner. Our true k -space is on a grid; a real scanner would have measured it along the spokes instead. Two parts:

1. The index conversion `coords + rows//2`.

Its Job: turn centre-offsets into array indices — the one place this conversion happens.

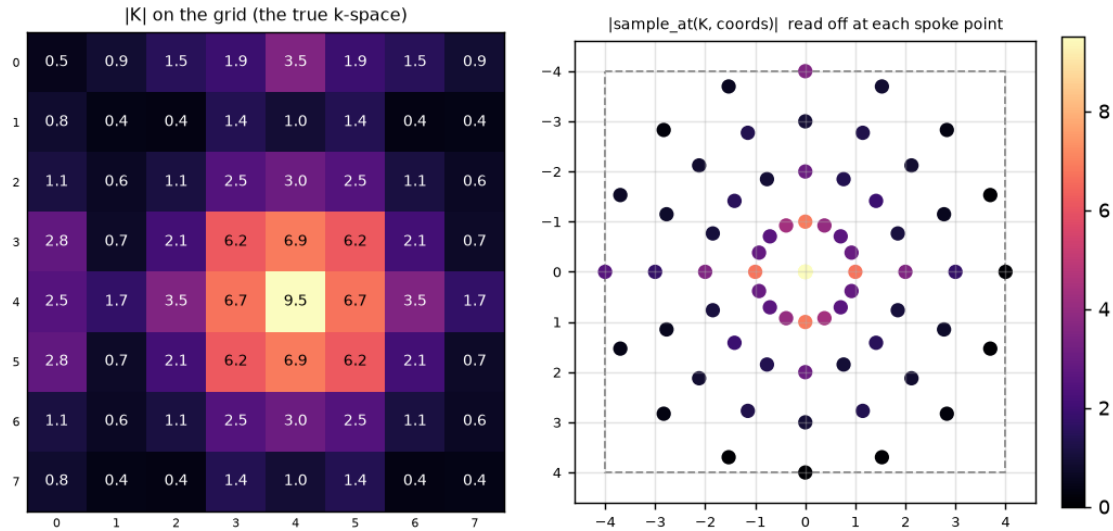
The Action: offset 0 becomes index 4; offset -4 becomes index 0; offset $+4$ becomes index 8.

2. The interpolation `map_coordinates(..., order=1)`.

Its Job: answer “what would K be at this off-grid point?”

The Action: bilinear interpolation from the four nearest grid cells. Real and imaginary parts are done separately because the function does not accept complex input; that is correct, because interpolation is linear.

```
idx = np.vstack([coords[:,0] + rows//2, coords[:,1] + cols//2]) # offsets -> indices
re = map_coordinates(k.real, idx, order=1) # bilinear
im = map_coordinates(k.imag, idx, order=1)
```



Reading the figure

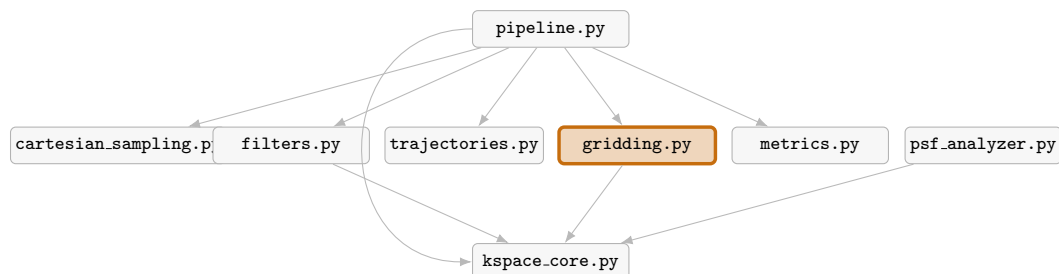
Left panel: the true $|K|$ on the grid, from Section 3.

Right panel: the value read off at each spoke point, coloured by magnitude. Bright at the hub, dark at the rim — the spokes are reading the same falloff the grid shows.

The check: the first spoke is horizontal through the centre, so it should read off row 4 of the left panel. It does: $[2.5, 1.7, 3.5, 6.7, \mathbf{9.5}, 6.7, 3.5, 1.7, 0.0]$ against $[2.5, 1.7, 3.5, 6.7, 9.5, 6.7, 3.5, 1.7]$.

(Note: the last sample on that spoke is 0 because offset +4 maps to index 8, one past the edge of an 8-wide array, and `map_coordinates` returns the fill value there. It is the only sample that falls outside.)

13 Putting the samples back on the grid



► `gridding.grid_accumulate(coords, values, (8,8), weights)`

This step shows true gridding — the heart of the non-Cartesian path. Each scattered sample is *spread* onto its four neighbouring grid points and *added* there. Three parts:

1. The four-corner split (the dy/dx loops).

Its Job: decide how much of a sample each neighbouring grid point receives.

Corner (0,0) — top-left: share $(1 - f_y)(1 - f_x)$.

Corner (0,1) — top-right: share $(1 - f_y) f_x$.

Corner (1,0) — bottom-left: share $f_y(1 - f_x)$.

Corner (1,1) — bottom-right: share $f_y f_x$. The four shares always sum to 1.

2. The accumulator `np.add.at`.

Its Job: sum every contribution that lands on the same cell, instead of overwriting.

The Action: when eight spokes all hit the centre cell, it records all eight.

3. The optional weights `values*weights`.

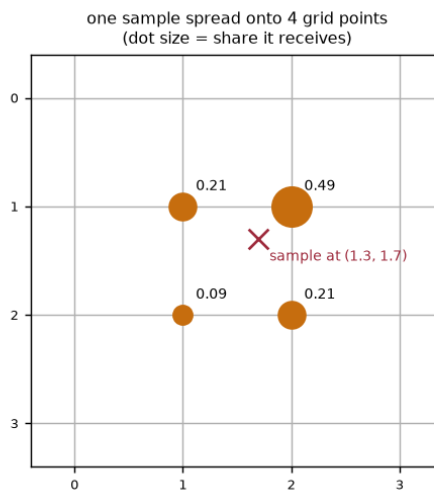
Its Job: apply density compensation *before* accumulating.

The Action: hub samples are scaled down by their tiny ramp weight, so their eight-fold pile-up is cancelled.

The maths

$$G[y_0 + \delta_y, x_0 + \delta_x] += v \cdot w \cdot (\delta_y f_y + (1 - \delta_y)(1 - f_y)) (\delta_x f_x + (1 - \delta_x)(1 - f_x))$$

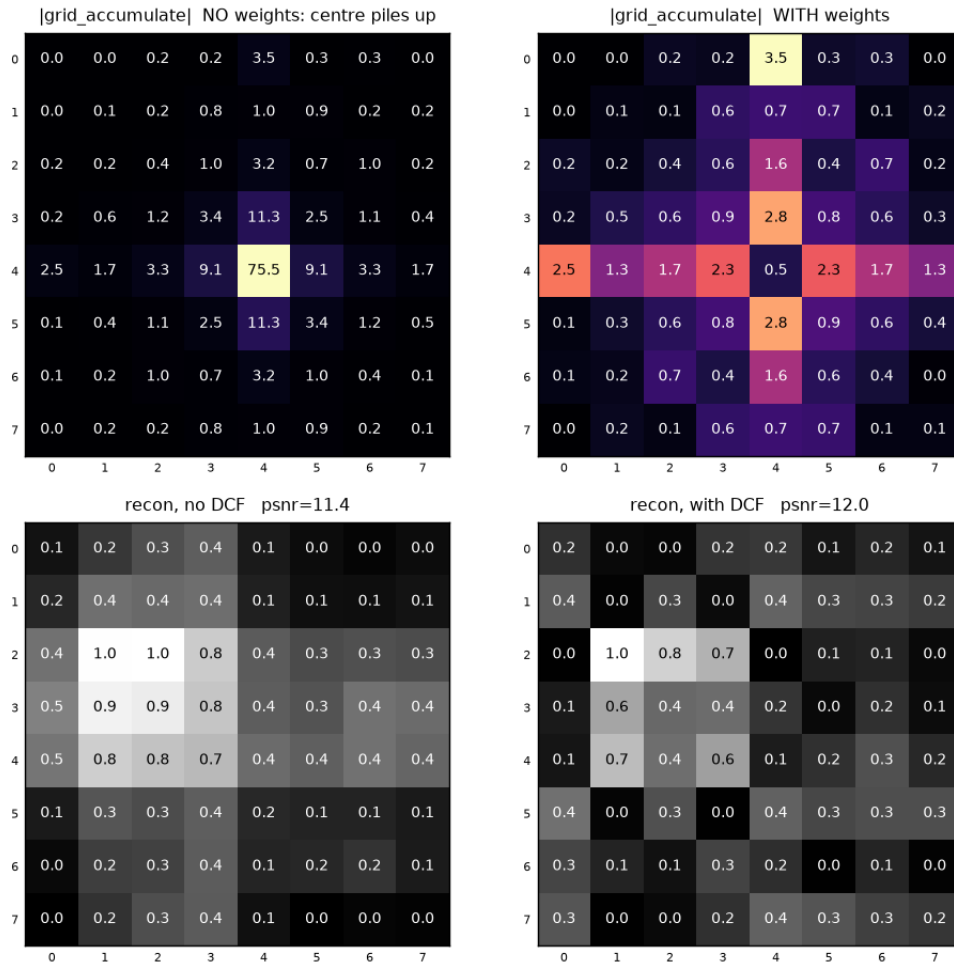
for $\delta_y, \delta_x \in \{0, 1\}$, where $f_y = y - \lfloor y \rfloor$.



The example: a sample at $(1.3, 1.7)$ gives 0.21 of itself to the top-left point, 0.49 to the top-right, 0.09 bottom-left, 0.21 bottom-right.

The edge case: a sample sitting exactly on a grid point gives that point everything and the other three nothing.

```
for dy in (0,1):
    for dx in (0,1):
        wgt = (fy if dy else 1-fy) * (fx if dx else 1-fx)
        np.add.at(out, (y0+dy, x0+dx), v*wgt) # add.at, NOT +=
```



Reading the figure

Top-left panel (no weights): the centre cell reads **75.5**. The true value is 9.5. Eight spokes each dropped 9.5 on the same cell and `np.add.at` faithfully summed them: $8 \times 9.5 \approx 76$. That is over-counting, made visible.

Top-right panel (with weights): the centre has been scaled back down and the rim values are now the largest. The ramp did its job.

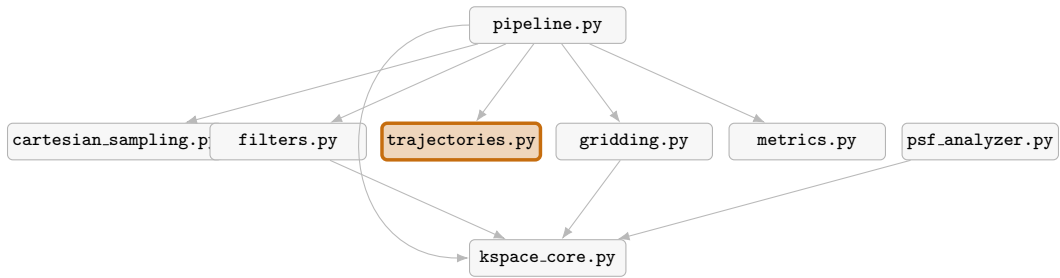
Bottom row: the two reconstructions. On 8×8 they differ only modestly (11.4 $\rightarrow$ 12.0 dB), because with 9 samples per spoke the ramp's floor is coarse.

Why `np.add.at` and not `out[y,x] += v`

With fancy indexing, if two samples target the same cell only the *last* write survives. The centre would read 9.5, not 75.5 — there would be no over-counting, and no reason for density weights to exist. `np.add.at` accumulates every contribution. It is the whole difference between gridding and interpolation.

(Note: the small DCF gain here is a scale effect, not a bug. Section 15 shows the same code on 256×256 , where the gain is twelve decibels.)

14 The other trajectory: spiral



► `trajectories.spiral((8,8), n_interleaves=4, n_turns=2, n_samples=40)`

This step shows the spiral trajectory. It feeds the same chain as radial — `sample_at`, then `grid_accumulate` — and only the coordinates change. Three parts:

1. The growing radius $r = k_{\max} \cdot t$.

The Action: as t runs $0 \rightarrow 1$, r runs $0 \rightarrow 4$. The arm starts at the centre and moves outward.

2. The sweeping angle $\text{base} = 2\pi \cdot n_{\text{turns}} \cdot t$.

The Action: two full rotations over the same t . Radius and angle grow together, which is what makes a spiral.

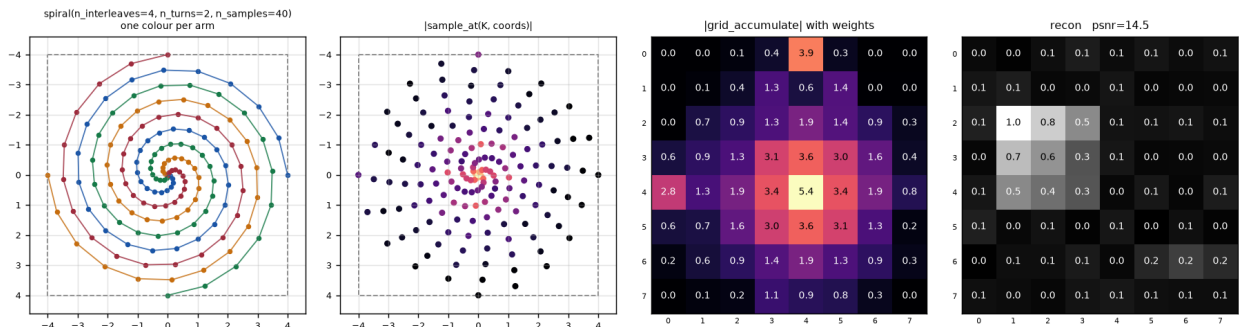
3. The interleaves (the for i loop).

Its Job: cover k -space with more than one arm, because a real scanner cannot acquire a whole spiral in one shot.

The Action: four copies of the same arm, each rotated by $2\pi i/4 = 90^\circ$. Same ramp weight as radial.

The maths

$$r(t) = 4t, \quad \theta_i(t) = 2\pi \cdot 2 \cdot t + \frac{2\pi i}{4}, \quad t \in [0, 1], \quad i = 0..3$$



Reading the figure

Panel 1: four arms, one colour each, all starting at the centre. This is what “interleaves” means — rotated copies fired one after another.

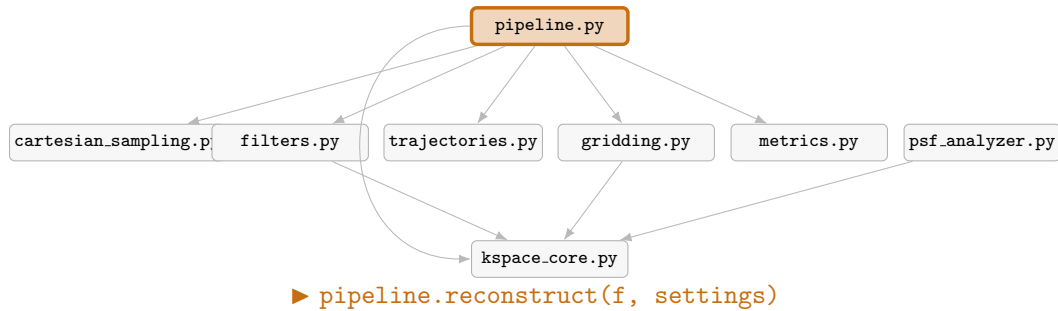
Panel 2: the sampled values. Bright near the hub, dark toward the rim. 160 samples in total.

Panel 3: the gridded k -space, with weights applied.

Panel 4: PSNR 14.5 — the best of any 8×8 method here, because 160 samples cover 64 cells far more densely than the 72 radial ones.

(Note: using $|r|$ as the spiral weight is an approximation — the exact density for an Archimedean spiral is messier — but the ramp is the standard first cut and it works. Worth saying so in your report.)

15 The coordinator



This step shows the one function the GUI will call. It runs everything above in a fixed order and hands back one dictionary. Two parts:

1. The settings merge `{**DEFAULTS, **settings}`.

Its Job: let the GUI send only the knobs it changed.

The Action: any key not supplied falls back to its default, so a partial dict is always safe.

2. The chain.

Path A — `sampling="cartesian"`: `make_mask` → `multiply`.

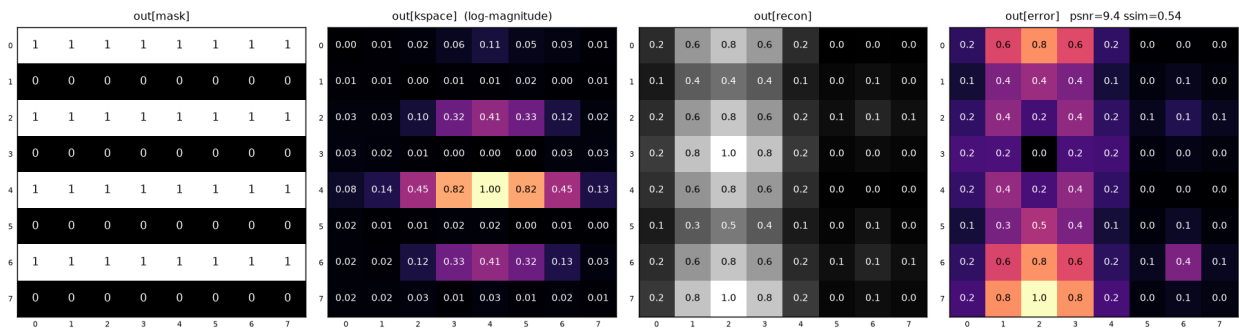
Path B — `sampling="radial" or "spiral"`: `trajectory` → `sample_at` → `grid_accumulate`.

Both paths then: `apply_window` → `add_noise` → `from_kspace` → `normalise` → `error_map` → `compute_metrics`.

```

out = reconstruct(f, {"sampling": "cartesian", "mask": "uniform", "R": 2,
                     "window": "hamming", "noise": 0.05, "seed": 1})

```



Reading the figure

The returned dict has five keys. Four are panels the GUI draws:

`out["mask"]`: the 0/1 mask, or `None` for non-Cartesian.

`out["kspace"]`: log-magnitude of the sampled k -space, already in $[0, 1]$.

`out["recon"]`: the reconstructed image, normalised.

`out["error"]`: $|f - \hat{f}|$ for the heatmap.

`out["metrics"]`: the fifth key — the readout. Here PSNR 9.4, SSIM 0.54.

(Note: noise is added after the window, because a real scanner's noise enters at the receiver and is therefore not itself apodized. The order in the chain is not arbitrary.)

Every setting, same grid

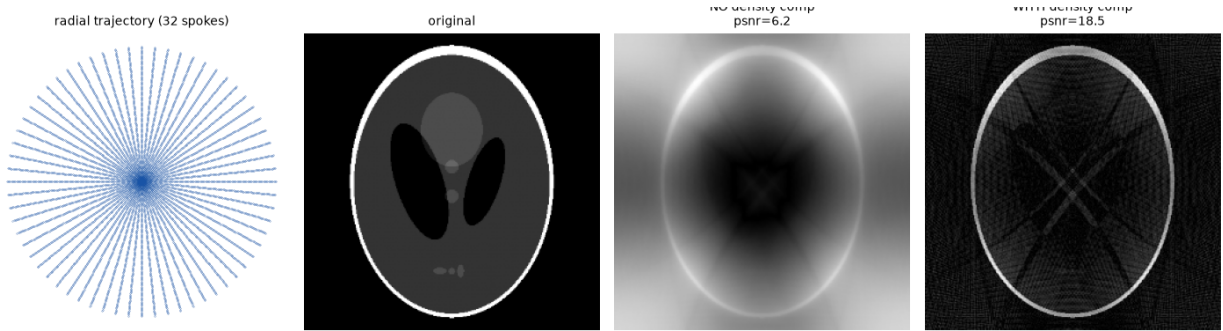
settings	PSNR (dB)	SSIM
R=1 (nothing removed)	328.9	1.000
R=2	8.4	0.611
R=2, window=hamming	9.5	0.540
R=2, noise=0.1	8.7	0.608
radial, n_spokes=8	10.7	0.460
radial, n_spokes=8, density_comp=False	10.6	0.519
spiral, n_interleaves=4	14.7	0.777

The identity row: $R = 1$ at 329 dB means the whole chain, with nothing switched on, reproduces the input to machine precision. That is Gate 6.

The damage rows: every setting below it loses quality, as it should. The GUI's sliders walk up and down this table.

16 The same thing at real size

This final figure shows the radial path on the 256×256 phantom with 128 spokes — the same `radial` → `sample_at` → `grid_accumulate` → `from_kspace` chain as Sections 10–12, at the size the app actually runs.



Reading the figure

Panel 1: the trajectory. Visibly dense at the hub.

Panel 3 (no weights): the over-counted centre swamps everything into a glare blob at 6.2 dB.

Panel 4 (with weights): the anatomy returns at 18.5 dB. Nothing changed except the weights.

The scale effect: on the 8×8 grid that gap was half a decibel. Here it is twelve. Long spokes give the ramp room to work.

(Note: two effects only show properly at this size — Gibbs ringing, which needs room for ripples, and density compensation, which needs long spokes. The 8×8 grid makes every number readable; the phantom makes the physics visible.)

To regenerate every figure after changing a function:

```
python docs/walkthrough/make_figs.py
```